Reporte Técnico de Actividades Práctico-Experimentales Nro. 002

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Juan Pablo Jumbo Orellana Luis Alejandro Rodríguez Luzón Jonathan Gerardo Contento Cuenca Sdier Emanuel Roblez Roblez
Asignatura	Diseño de Circuitos
Ciclo	Segundo (2.°) – Paralelo A
Unidad	Unidad 1. Análisis de lógica combinacional con diseño clásico
Resultado de aprendizaje de la unidad	R1. Diseña e implementa circuitos combinacionales básicos utilizando técnicas de diseño clásico, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	002
Título de la Práctica	Formas canónicas y simplificación algebraica
Nombre del Docente	Ing. Luis Darío Sinche Cueva, Mg.
Fecha	20 de octubre de 2026
Horario	07:30 – 09:30
Lugar	Laboratorio de computación de la Carrera de Computación
Tiempo planificado en el Sílabo	2 horas

2. Objetivo(s) de la Práctica

O1. Traducir un requerimiento enunciado en lenguaje natural a una especificación lógica sin ambigüedad —tabla de variables y tabla de verdad— y obtener sus formas canónicas de suma de productos (SOP) y producto de sumas (POS).

O2. Implementar en Logisim-evolution el circuito canónico y su versión simplificada algebraicamente, comprobar que ambos cumplen la misma tabla de verdad y comparar su costo en compuertas y en circuitos integrados de la familia 74LS.

3. Materiales, Reactivos, Equipos y Herramientas

- No aplica: la práctica se desarrolla íntegramente en simulación.
- Hojas de datos de los circuitos integrados SN74LS04 (seis inversores), SN74LS08 (cuatro AND de 2 entradas), SN74LS11 (tres AND de 3

entradas) y SN74LS32 (cuatro OR de 2 entradas), disponibles en el aula virtual.
- Tabla de consulta de postulados y teoremas del álgebra de Boole, disponible en el aula virtual.
- Hoja de registro de la práctica (Tablas 2 a 6), impresa o en el cuaderno.
- Computador del laboratorio o computador personal (Windows, macOS o Linux).
- Simulador Logisim-evolution (software libre, licencia GPL v3), instalado localmente; funciona sin conexión a internet.
- Lector de PDF para consultar las hojas de datos.
- Aula virtual (EVA) de la asignatura, para descargar recursos y entregar el informe.

4. Procedimiento / Metodología Ejecutada

La práctica se realiza en los grupos de trabajo de la asignatura (Grupos 1 a 8). Cada grupo resuelve el requerimiento que le asigna la Tabla 1, distinto para cada grupo, y presenta su propio trabajo. Se aplica la estrategia de diseño verificado antes de implementado: todo el diseño de las Partes A y B se realiza en papel antes de abrir el simulador, y el simulador se usa para comprobar ese diseño, no para obtenerlo.

Inicio:

1. El docente recupera la clase anterior: un circuito y una expresión son la misma cosa escrita de dos maneras; minitérminos y maxitérminos; formas canónicas SOP y POS; teoremas de adyacencia, absorción e idempotencia.

2. Se socializan los objetivos, la estructura del informe, la rúbrica (Anexo 1) y la política de uso declarado de inteligencia artificial.

3. Advertencia: Logisim-evolution incluye una herramienta de análisis combinacional capaz de construir y minimizar circuitos de forma automática. En esta práctica esa herramienta se usa únicamente para verificar (Parte D). Un circuito generado automáticamente se califica con 0 en el criterio de montaje.

4. Cada grupo abre Logisim-evolution, crea un proyecto nuevo y lo guarda como APE02_GrupoN.circ (N = número de grupo). Con Proyecto → Añadir circuito, crea dos subcircuitos: «canónica» y «simplificada».

Parte A. Del requerimiento a la especificación (en papel)

1. Lea el requerimiento asignado e identifique las tres entradas y la salida. Complete la Tabla 2 indicando qué significa el 0 y qué significa el 1 en cada señal. El valor 1 debe corresponder al estado que nombra la variable (por ejemplo, «D = docente presenta tarjeta»).

2. Si el enunciado admite más de una lectura, escriba en el informe la lectura que adopta y justifíquela.

3. Construya la tabla de verdad (Tabla 3) en orden binario, con la primera variable de la Tabla 2 como bit más significativo, y numere las filas de 0 a 7.

Parte B. Formas canónicas y simplificación (en papel)

1. A partir de las filas con F = 1, escriba la función en notación compacta Σm( ) y su forma canónica SOP.

2. A partir de las filas con F = 0, escriba la función en notación compacta ΠM() y su forma canónica POS.

3. Según el número de unos y de ceros de la tabla, indique qué forma canónica es más corta y elija una para simplificar.

4. Simplifique algebraicamente la forma elegida. Registre cada paso en la Tabla 4 con el postulado o teorema aplicado. Continúe hasta que no queden pares de términos que difieran en una sola variable.

5. Compruebe en papel la expresión simplificada en al menos dos filas de la Tabla 3, una con F = 1 y otra con F = 0.

Parte C. Montaje en Logisim-evolution

1. En el subcircuito «canonica», monte la forma canónica SOP con compuertas de la biblioteca Compuertas (Gates). Use compuertas NOT para obtener los complementos y ajuste la propiedad «Número de entradas» de las AND y OR cuando necesite más de dos. Rotule los pines de entrada con el nombre de sus variables y el pin de salida como F.

2. En el subcircuito «simplificada», monte la expresión simplificada obtenida en la Parte B, con los mismos rótulos.

3. Con la herramienta Pulsar, recorra las ocho combinaciones de entrada en orden binario en ambos subcircuitos y registre los resultados en la Tabla 5.

4. Capture una imagen de cada subcircuito en una combinación con F = 1, en la que se vean los rótulos y el estado de la salida.

Parte D. Verificación con Analizar circuito

1. Con el subcircuito «canónica» abierto, seleccione Proyecto → Analizar circuito. En la pestaña «Tabla», compare la tabla generada con su Tabla 3 y capture la pantalla. Repita con el subcircuito «simplificada».

2. En la pestaña «Expresión», observe la expresión que muestra el simulador para cada subcircuito. Si está escrita de otra forma o en otro orden que la suya, demuestre en el informe que ambas son equivalentes.

3. No utilice el botón «Construir circuito»: el circuito debe ser el que usted diseñó.

Parte E. Recuento de costo

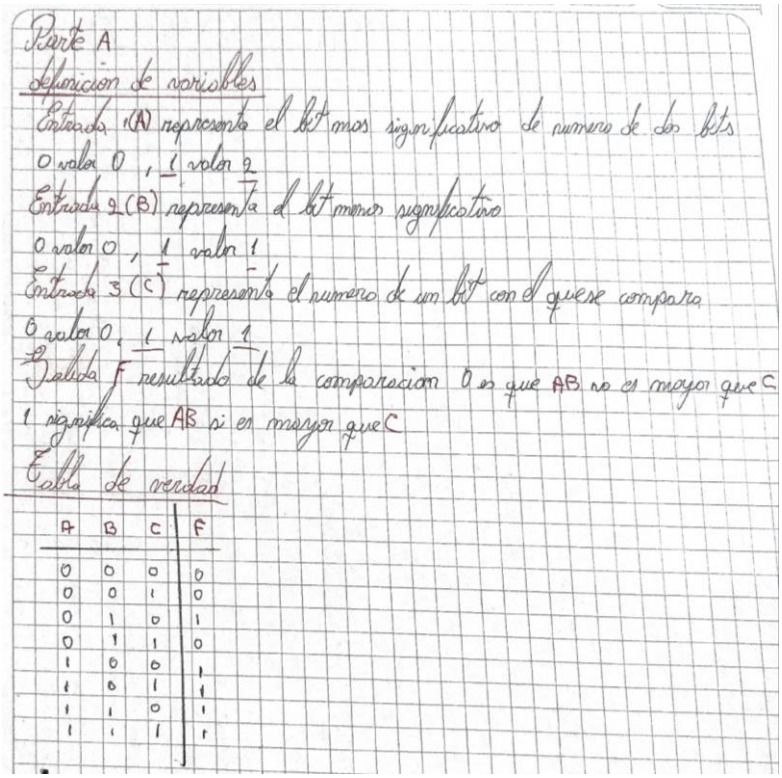
1. Complete la Tabla 6 para las dos versiones montadas y, solo en papel, para la forma canónica POS. Para el número de circuitos integrados consulte las hojas de datos: el 7404 trae seis inversores, el 7408 cuatro AND de 2 entradas, el 7411 tres AND de 3 entradas y el 7432 cuatro OR de 2 entradas. Una OR de 3 entradas se arma con dos OR de 2 entradas.

2. Indique en la última columna cuántos niveles de compuertas atraviesa la señal, desde la entrada hasta la salida, en el camino más largo.

5. Resultado

Drive:
https://drive.google.com/file/d/1VCFFcQNL_ztB37Tiv0BwO53lqlZgcHr2/view?usp=sharing

Parte A. Del requerimiento a la especificación (en papel)



Variables

Entrada 1 (A): Representa el bit más significativo del número de dos bits. (0 significa valor cero, 1 significa valor dos).

Entrada 2 (B): Representa el bit menos significativo. (0 significa valor cero, 1 significa valor uno).

Entrada 3 (C): Representa el número de un bit con el que se compara. (0 significa valor cero, 1 significa valor uno).

Salida F: Indica el resultado de la comparación. (0 significa que AB no es mayor que C, 1 significa que AB sí es mayor que C)

. Tabla 2. Definición de variables

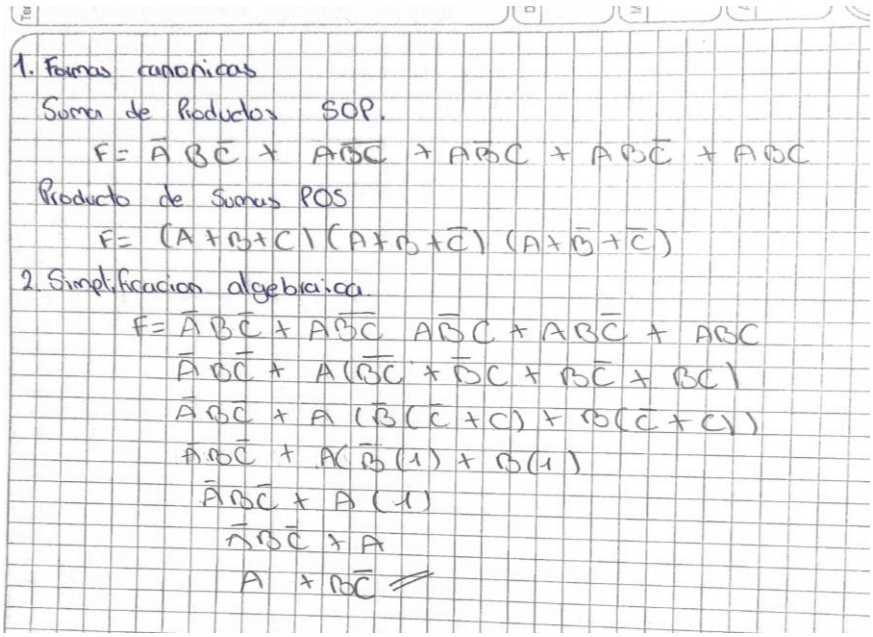
Señal	**Significado**	**0 significa**	**1 significa**
A	Bit más significativo de AB	Valor 0	Valor 2
B	Bit menos significativo de AB	Valor 0	Valor 1

C	Número de 1 bit a comparar	Valor 0	Valor 1
F	Resultado: ¿Es AB mayor que C?	Falso (AB no es mayor)	Verdadero (AB > C)

Tabla 3. Tabla de verdad del requerimiento.

Fila	Entrada 1 (A)	Entrada 2 (B)	Entrada 3 (C)	Salida F
0	0	0	0	**0**
1	0	0	1	**0**
2	0	1	0	**1**
3	0	1	1	**0**
4	1	0	0	**1**
5	1	0	1	**1**
6	1	1	0	**1**
7	1	1	1	**1**

Parte B. Formas canónicas y simplificación (en papel)



3. Según el número de unos y de ceros de la tabla, indique qué forma canónica es más corta y elija una para simplificar

La forma canónica más corta es el **Producto de Sumas (POS / Maxitérminos)**, ya que la función solo produce un valor de **0** en **3 filas** de la tabla de verdad, mientras que produce un **1** en **5 filas** (SOP / Minitérminos).

Se eligió para simplificar la SOP.

Paso	Expresión	Postulado o teorema aplicado
1	$F = \overline{A}B\overline{C} + A\overline{B\overline{C}} + A\overline{B}C + AB\overline{C} + ABC$	Expresión canónica en suma de productos (SOP)
2	$F = \overline{A}B\overline{C} + A\left(\overline{B\overline{C}} + \overline{B}C + B\overline{C} + BC\right)$	Factor común de A (Ley distributiva)
3	$F = \overline{A}B\overline{C} + A\left(\overline{B}(\overline{C} + C) + B(\overline{C} + C)\right)$	Factor común de $\overline{B} y B$ (Ley distributiva)
4	$F = \overline{A}B\overline{C} + A\left(\overline{B}(1) + B(1)\right)$	Ley del complemento $(\overline{C} + C = 1)$
5	$F = \overline{A}B\overline{C} + A(\overline{B} + B)$	Elemento neutro del producto $(X \cdot 1 = X)$
6	$F = A + B\overline{C}$	Ley del complemento $(\overline{B} + B = 1)$ y ley de absorción $\left(A + \overline{AB\overline{C}} = A + B\overline{C}\right)$

Comprobación de la expresión $F = A + BC'$

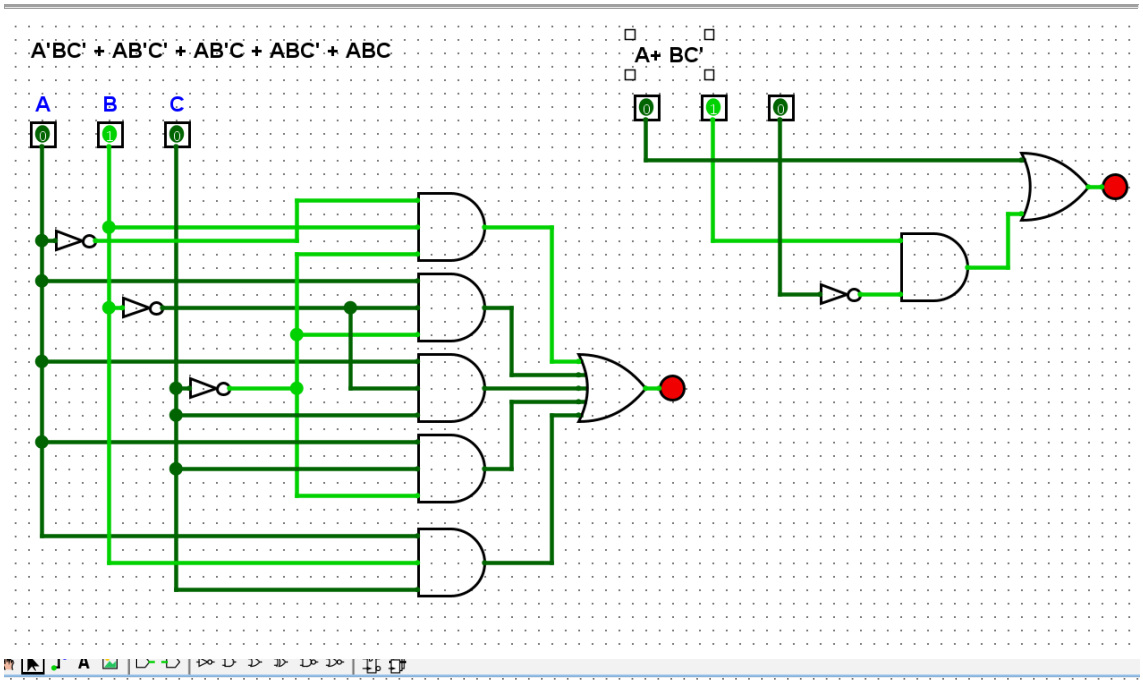
- **Con una fila donde F=1 (Fila 4: A=1, B=0, C=0):**

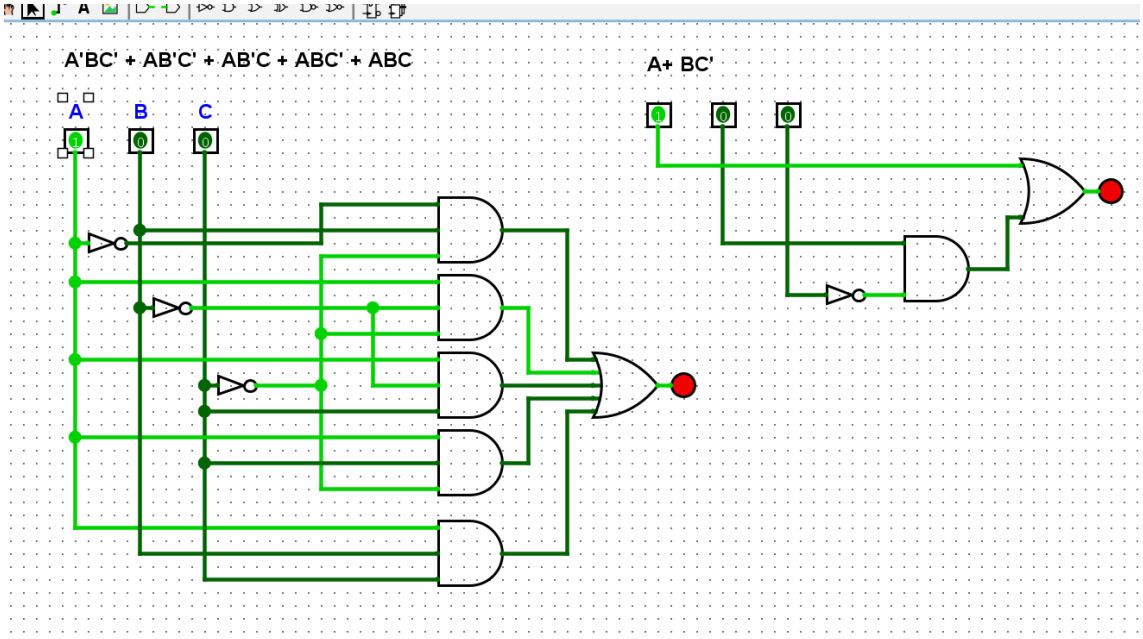
$F = 1 + (0 \cdot 0') = 1 + 0 = 1$. Se cumple.

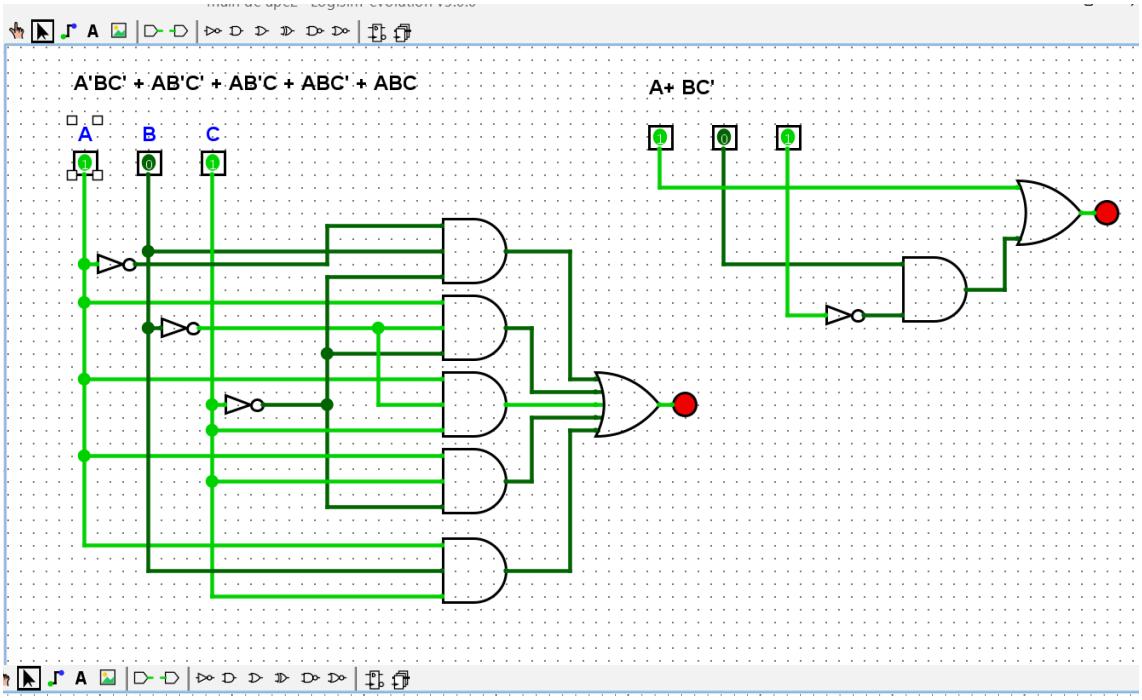
- **Con una fila donde F=0 (Fila 3: A=0, B=1, C=1):**

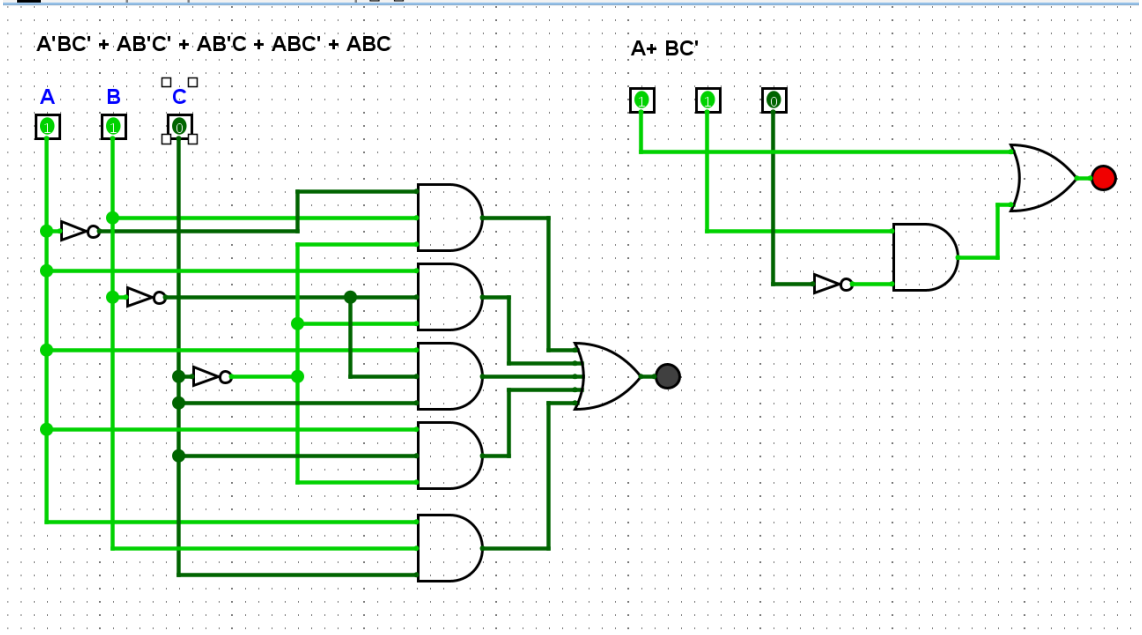
$F = 0 + (1 \cdot 1') = 0 + (1 \cdot 0) = 0$. Se cumple.

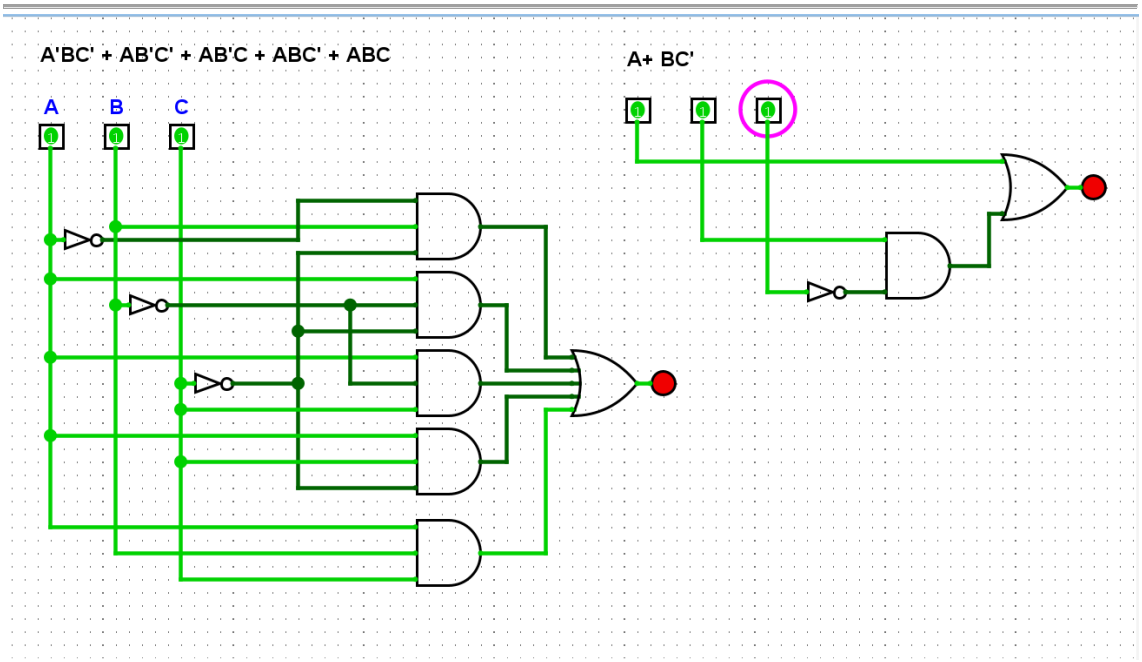
Parte C. Montaje en Logisim-evolution


A'BC' + AB'C' + AB'C + ABC' + ABC
A+ BC'
A
B
C


A'BC' + AB'C' + AB'C + ABC' + ABC
A+ BC'
A
B
C


A'BC' + AB'C' + AB'C + ABC' + ABC
A+ BC'
A
B
C


A'BC' + AB'C' + AB'C + ABC' + ABC
A+ BC'
A
B
C



Fila	Entradas	F especificada (Tabla 3)	F canónica	F simplificada
0	000	0	0	0
1	001	0	0	0
2	010	1	1	1
3	011	0	0	0
4	100	1	1	1
5	101	1	1	1
6	110	1	1	1
7	111	1	1	1

Parte D. Verificación con Analizar circuito

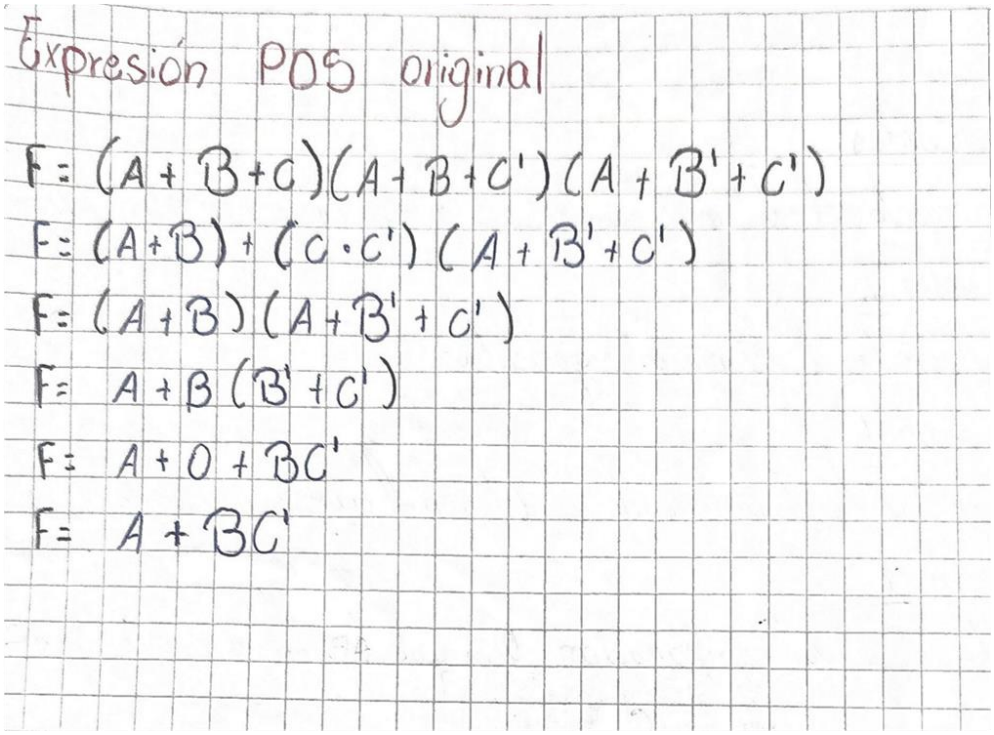
Simulacion de la canonica y simplificada

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Demostración de equivalencia: El simulador (Logisim-evolution) puede mostrar las expresiones factorizadas de otra manera o en otro orden debido a su algoritmo interno de optimización, pero al evaluar ambas expresiones mediante su tabla de verdad (comprobando fila por fila los resultados), se demuestra que arrojan exactamente los mismos valores de salida para cada una de las 8 combinaciones posibles. Por lo tanto, la función lógica obtenida por el estudiante y la mostrada por el simulador son **estrictamente equivalentes**.

Parte E. Recuento de costo



Versión	Compuertas por tipo	Total, de compuertas	CI 74LS necesarios	Niveles lógicos
SOP canónica	3 NOT, 5 AND (3 ent.), 4 OR (2 ent.)	12	4 (1x7404, 2x7411, 1x7432)	5
POS canónica (papel)	2 NOT, 6 OR (2 ent.), 1 AND (3 ent.)	9	4 (1x7404, 2x7432, 1x7411)	4
Versión	**Compuertas por tipo**	**Total, de compuertas**	**CI 74LS necesarios**	**Niveles lógicos**
Simplificada	1 NOT, 1 AND (2 ent.), 1 OR (2 ent.)	3	3 (1x7404, 1x7408, 1x7432)	3

6. Preguntas de Control

1. Explique, con un minitérmino de su tabla, por qué un minitérmino vale 1 en una sola fila. Haga lo mismo con un maxitérmino y la fila en la que vale 0.

- **Minitérmino** $\left(m_2 = \overline{AB\overline{C}}\right)$**:** Corresponde a la fila 2 (A=0, B=1, C=0). Al ser una compuerta AND lógica, todas sus entradas deben ser 1 para que la salida sea 1. Sustituyendo los valores de la fila 2, obtenemos $\overline{0} \cdot 1 \cdot \overline{0} = 1 \cdot 1 \cdot 1 = 1$. En cualquier otra fila, al menos una variable cambiará de valor, forzando un 0 en la multiplicación y anulando el minitérmino.
- **Maxitérmino** $\left(M_1 = A + B + \overline{\{C\}}\right)$**:** Corresponde a la fila 1 ($A = 0$, $B = 0$, $C =$). Al ser una compuerta OR, todas sus entradas deben ser 0 para que la salida sea 0. Sustituyendo, obtenemos $0 + 0 + \overline{1} = 0 + 0 + 0 = 0$.

En cualquier otra fila, al menos una variable aportará un 1, haciendo que la suma$0 + 0 + \overline{1} = 0 + 0 + 0 = 0$ lógica sea 1.

2. Si la columna F de su tabla se invirtiera (los 1 pasan a 0 y viceversa), ¿cuál sería el nuevo Σm? ¿Qué relación tiene con el ΠM de su función original?

- Si se invierten los valores, las filas que valían 0 (filas 0, 1 y 3) ahora valdrán 1.
- El nuevo conjunto de minitérminos será $\Sigma m(0,1,3)$.
- **Relación:** Este nuevo Σm utiliza exactamente los mismos índices que el Producto de Maxitérminos original $(\Pi M(0,\ 1,\ 3))$. Esto evidencia el Teorema de De Morgan, demostrando que la función negada es equivalente a la intersección de los maxitérminos de la función original.

3. Según el número de unos y de ceros, ¿qué forma canónica era más corta para su requerimiento? ¿La expresión mínima que obtuvo tiene forma de suma de productos o de producto de sumas?

- La forma canónica más corta era el **Producto de Sumas (POS)**, dado que la tabla de verdad solo produce 0 en tres filas, frente a cinco filas con 1 (SOP).
- La expresión mínima obtenida $(F = A + B\overline{C})$ tiene forma de **Suma de Productos (SOP)**.

4. En su simplificación, ¿hubo algún par de términos que parecía combinable, pero difería en más de una variable? Explique por qué no puede reducirse.

En la simplificación de la forma POS, al comparar el término de la fila 0 (A + B + C) con el de la fila 3 (A + B' + C'), a primera vista podría parecer intuitivo intentar agruparlos por el hecho de que ambos comparten la variable A. Sin embargo, no pueden reducirse porque difieren en **dos variables al mismo tiempo** (B cambia a B' y C cambia a C').

El Teorema de Adyacencia del álgebra de Boole exige de forma estricta que dos términos difieran en **exactamente una variable** para que su suma o producto cancele el estado de dicha variable. Al existir dos diferencias simultáneas, no es posible factorizar la expresión de modo que los complementos se anulen entre sí.

5. ¿Cuántas compuertas y cuántos circuitos integrados ahorró la versión simplificada? Además del costo, mencione otra ventaja de reducir el número de niveles lógicos.

Ahorro frente a SOP Canónica: Se ahorraron **9 compuertas** (de 12 a 3) y **1 circuito integrado** (de 4 chips a 3).

Ahorro frente a POS Canónica: Se ahorraron **6 compuertas** (de 9 a 3) y **1 circuito integrado** (de 4 chips a 3).

Ventaja de reducir los niveles lógicos: Aparte de abaratar costos y simplificar el cableado en la placa, la principal ventaja física es la **disminución del retardo de propagación (tiempo de respuesta)**. Cada compuerta electrónica tarda unos nanosegundos en conmutar su estado. Al reducir el camino más largo de 5 niveles lógicos a solo 3, la señal llega a la salida F mucho más rápido, lo que permite

trabajar a frecuencias de reloj más altas y disminuye la aparición de estados transitorios no deseados (*glitches*) en el circuito.

6. Suponga que en la Tabla 2 asigna el valor 1 al estado contrario de una de sus entradas. Rehaga la tabla de verdad y la expresión mínima. ¿Cambia el circuito? ¿Cambia el comportamiento del sistema?

Tomemos como ejemplo la entrada C: si ahora definimos que 1 significa que la entrada vale 0, y 0 significa que vale 1 (lógica negada, C_nueva = C'), la nueva especificación modifica el llenado de las columnas.

Tabla de verdad y expresión mínima: Las filas donde antes teníamos C=0 ahora se leen como C_nueva=1. La tabla ajusta sus combinaciones y la función simplificada pasa de ser F = A + B C' a **F = A + B * C_nueva**.

¿Cambia el circuito? Sí. En el esquema físico ya no se requiere la compuerta NOT para la señal C, ya que la variable ingresa directamente en el estado que necesita la compuerta AND.

¿Cambia el comportamiento del sistema? No. El comportamiento lógico y funcional del sistema en la vida real se mantiene exactamente igual: el comparador sigue determinando con total precisión si el número AB es mayor que C. Lo único que cambió fue la convención o interpretación del voltaje que entra por ese pin específico.

7. Conclusiones

1. La traducción del problema planteado en lenguaje natural a una tabla de verdad bien definida permitió eliminar cualquier ambigüedad en el diseño del comparador. A partir de este proceso, la obtención de las expresiones canónicas SOP y POS proporcionó la base matemática exacta para representar el comportamiento lógico del circuito.

2. La implementación realizada en Logisim-evolution confirmó que tanto el circuito canónico como la versión simplificada responden de forma idéntica ante las ocho combinaciones de entrada de la tabla de verdad. Esto valida que la simplificación algebraica preservó la lógica original del sistema sin alterar sus resultados.

3. El análisis de costo demostró la importancia de simplificar las expresiones booleanas antes de pasarlas a la práctica. La versión simplificada redujo drásticamente el número de compuertas y circuitos integrados 74LS necesarios,

lo que se traduce en un menor costo económico, menor cableado y una respuesta más rápida del circuito al reducir los niveles lógicos.

8. Declaración de Uso de Inteligencia Artificial

- **Herramienta utilizada:** Asistente de Inteligencia Artificial (Gemini / Modelo de Lenguaje de IA).
- **Finalidad del uso:** La herramienta se utilizó como tutor pedagógico para guiar la interpretación del requerimiento R2 (Comparador), verificar la estructuración de las tablas de variables y de verdad, revisar paso a paso la simplificación algebraica de la forma POS canónica, corroborar el recuento comparativo de costos de hardware (Tabla 6) y estructurar la fundamentación lógica de las preguntas de control.
- **Proceso de verificación:** Ningún resultado generado por la IA se asumió como definitivo sin comprobación previa. La tabla de verdad se validó manualmente en papel evaluando cada combinación de entradas de 0 a

7. La simplificación algebraica se comprobó sustituyendo valores en las filas con F=1 y F=0.El número de compuertas y circuitos integrados se verificó directamente contra las hojas de datos oficiales de los chips 74LS04, 74LS08, 74LS11 y 74LS32, y el comportamiento global se contrastó con las pautas de la guía de la práctica APE 02.

9. Bibliografía / Referencias

- Carpinelli, J. D. (2023). *An animated introduction to digital logic design*. New Jersey Institute of Technology.
- Garza, J. Á. (2025). *Electrónica digital I: Sistemas digitales*. Nuevo León, México. Recurso en línea.
- Logisim-evolution developers. (2024). *Logisim-evolution* [Software de código abierto]. GitHub. https://github.com/logisim-evolution/logisim-evolution
- Texas Instruments. (s. f.). *SN74LS04: Hex inverters* [Hoja de datos]. https://www.ti.com/product/SN74LS04
- Texas Instruments. (2017). *SN54LS08, SN74LS08: Quadruple 2-input positive-AND gates* [Hoja de datos]. https://www.ti.com/product/SN74LS08
- Texas Instruments. (s. f.). *SN74LS11: Triple 3-input positive-AND gates* [Hoja de datos]. https://www.ti.com/product/SN74LS11
- Texas Instruments. (s. f.). *SN74LS32: Quadruple 2-input positive-OR gates* [Hoja de datos]. https://www.ti.com/product/SN74LS32